

Yongjun Rong

Cherry Hill, NJ, USA, 08003 (US Citizen) **Email:** rongyj@hotmail.com **Tel:** 856-408-4027

Linkedin: <https://www.linkedin.com/in/rongyongjun/>, **Github:** <https://github.com/rongyj>

EDUCATION

Jun.,2006, Master of Computer Science, Texas Tech University, Lubbock, TX, USA

Feb.,1996, Master of Engineering, Shanghai JiaoTong University, Shanghai, China

Jul.,1991, Bachelor of Engineering, Changsha Railway University, Changsha, Hunan, China

SUMMARY:

Hands-on principal-level software engineer with deep backend and distributed-systems experience: designing, building and operating production services in Java/Spring Boot, Python and TypeScript/Node.js, from contract-first microservices on OpenAPI to event-driven systems on Kafka, MQ and AWS serverless. Currently owns the reference architecture and much of the implementation for an on-demand application platform serving hundreds of developers across more than 80 product teams at a global bank - the Spring Boot provisioning backend, the ECS/EKS and Istio service-mesh design, and the observability, secrets and TLS standards every onboarding service inherits. Four years building production GenAI systems on top of that: RAG over ChromaDB/Milvus, multi-agent orchestration with AutoGen, concurrent LLM invocation, and FastAPI services other engineering teams consumed rather than rebuilt. Earlier, lead architect and lead implementer of a commercial multi-cloud cost analytics platform on Spring Boot/ClickHouse, lead software engineer on Oracle's cloud integration connectivity SDK, and the author of Barclays' group-wide Microservices Architecture Handbook. Works day to day in Claude Code, GitHub Copilot and Cursor, sets the practice for how his teams use them, and packages recurring platform work as agent skills and subagents behind an MCP server. Operates at principal scope: sets the standards, writes the reference implementation, and mentors the engineers who extend it. US Citizen; fluent English, native Chinese.

SKILLS:

- Languages: Java, Python, TypeScript/JavaScript, Kotlin, Go, SQL, Bash, Terraform HCL.
- Backend and distributed systems: Spring Boot, FastAPI, Node.js; contract-first microservices on OpenAPI; event-driven architecture, CQRS, event sourcing; Kafka, IBM MQ, RabbitMQ, EventBridge/SQS/SNS; ClickHouse, PostgreSQL, DynamoDB; PySpark and Databricks.
- AWS and platform engineering: ECS/Fargate, EKS, Lambda, Step Functions, ALB, Route 53, ACM, RDS, S3, IAM/OIDC/IRSA, Secrets Manager, KMS; Istio service mesh with mesh-wide mTLS; multi-tenant isolation and quotas. Azure AKS and GCP exposure.
- Infrastructure as code: Terraform and Terragrunt module libraries, DRY multi-environment configuration; Puppet, Chef, Ansible; CI/CD on Jenkins, Bamboo and Bitbucket pipelines.
- GenAI engineering: RAG over ChromaDB and Milvus, embeddings and retrieval tuning, multi-agent orchestration (AutoGen), prompt engineering, async concurrent LLM invocation, model-backed APIs on FastAPI.
- AI-assisted engineering: Claude Code, GitHub Copilot and Cursor daily; recurring work automated as agent skills, subagents and MCP servers; sets the team's trust and review boundaries for generated code.
- Engineering practice: Clean Architecture, SOLID, JUnit with Docker test containers, design and code review; Dynatrace, Splunk, Prometheus/Grafana/Jaeger, CloudWatch; JVM heap and GC tuning.
- Technical leadership: authored Barclays' group-wide Microservices Architecture Handbook, chaired a ~30-member technology strategy working group setting invest/divest positions across ~200 products, and mentors engineers on service mesh, IaC and agentic workflows.

HOBBIES

- Pickleball (~4.0 player)
- IoT or Robot programming (Have iRobot and MIPOSaur at home. Programming against iRobot create SDK and MIPOSaur SDK.)
- Playing with cutting-edge technologies (genAI/Autogen/Multiagents, Docker, Go, Kafka, Kubernetes and Amazon CloudFormation, JavaScript, AngularJS, ReactiveJS, Node.js)

WORK EXPERIENCE SUMMARY:

Employments	Major Accomplishments & Technologies
Oct.,2025 – Current Intuitive.ai Cherry Hill, NJ (WFH),USA Principal Cloud and Platform Architect (JPMC)	- On Demand Environment (ODE) backend (Java Spring Boot) infrastructure (ECS) design and implementation - EKS Service Mesh (Istio) on-demand framework Design and Implementation - Design and implement AWS Secrets (ASM) solution for Java Spring Boot Microservices - Proprietary Environment As Code (EAC/IAC) self-teaching, mentoring and troubleshooting - Design and implement Splunk and Dynatrace onboarding for Spring Boot ECS Microservices IaC, Terraform, AWS/EKS/Service Mesh/Istio, AI-assisted development (Claude Code, GitHub Copilot, Cursor), AI agents / MCP / agentic workflow automation, TLS Certificate,
Feb.,2025 – Oct.,2025 Biophy Cherry Hill, NJ (WFH),USA Director of Cloud Architecture	- AI-Powered Data Ingestion and Scraping System for RAG - Custom AWS CloudWatch Alerting System with EventBridge, Lambda, SQS and SNS IaC, Terraform/Terragrunt, AWS/EC2/EBS/S3/VPC/ALB/Security/R53/IAM/Secrets Manager/Lambda/CloudWatch/EventBridge/SQS/SNS, Python/Django/Pydantic, openAI API/LangChain, SOLID Principles, Clean Architecture,
Dec.,2021 – Feb.,2025 McKinsey & Company Cherry Hill, NJ (WFH),USA Senior Cloud Engineer II (Associate)	- Using genAI (openAI) with RAG (ChromaDB, Milvus) to accelerate the development of data pipelines running on pyspark and databricks - Using genAI (openAI) with multi-agents to accelerate the migration of mainframe to modern technologies - Migration/Refactoring of Mainframe (IBM MQ) EDA system to AWS by using EventBridge/Lambda/Step Function/CloudWatch/EKS/SQS/SNS...etc. genAI/openAI/Langchain/RAG/Multi-agents/prompts tuning, AI-assisted development (GitHub Copilot, Cursor), VectorStore/ChromaDB/Milvus, Python/Flask/FastAPI/Pydantic/Jupyter, Databricks/pyspark, Typescript/JavaScript/NodeJs, Event Driven Architecture (EDA), Terraform, Microservices, Java/SpringBoot/NodeJs/Python/Flask/FastAPI, EventBridge/CloudWatch/Lambda/Step Functions, DynamoDB/RDS/Oracle, Docker/K8s/EKS/AKS, Json Schema, JOI, OpenAPI/Swagger,
Dec.,2020 – Dec., 2021 Yotascale Cherry Hill, NJ (WFH),USA Staff Engineer	Lead Architect and Lead Implementation Engineer for Cloud(AWS/Azure/GCP) Cost Analytics/Optimization/Management platform OpenAPI/Swagger, Microservice, Java/Kotlin/SpringBoot, Kubernetes/EKS/AKS, Istio, Terraform, CQRS, Event Store, Event Driven Architecture (EDA), Kafka/ActiveMQ/IBM MQ, Kafka Stream, Docker, AWS(EC2/IAM/S3/ECS/EKS/ELB/Lambda/RDS/VPC/R53/Secret Manager/AutoScaling/SES..etc), Puppet/Chef/Ansible, Git/Maven/Jenkins, Python/Bash/Go, ELK/AppDynamic/Datadog,
Mar.,2020 – Nov., 2020 Gigster Inc. Cherry Hill, NJ,USA Technical Infrastructure/Devops Architect	Images manipulation/detection software evaluation system Microservice, NodeJs, Kubernetes/EKS, Terraform, RabbitMQ/ActiveMQ, Docker, AWS(EC2/IAM/S3/EKS/ALB/RDS/VPC/R53/AutoScaling/SES/ECR..etc), Puppet/Chef/Ansible, Git/Maven/Jenkins, JavaScript,
Jun.,2017 – Feb., 2020 Barclays Wilmington, DE,USA VP,Lead Architect Engineer	- Delivery Lead for strategic distributed system technology domain - Group-wide Cloud Adoption for Contact Center, Merchant Services and Fraud Terraform, CQRS, Event Store, Event Driven Architecture (EDA), Kafka/ActiveMQ/IBM MQ, Kafka Stream, Docker, AWS(EC2/IAM/S3/ECS/EKS/ELB/Lambda/RDS/VPC/R53/Secret Manager/AutoScaling/SES..etc), Puppet/Chef/Ansible, Git/Maven/Jenkins/Bamboo, Python/Ruby/Scala/Bash/Go, ELK/AppDynamic, DynamoDB/MongoDB/ElasticCache/GridGain, Linux/Windows/AIX/Solaris, aPaaS/VMWare/Citrix,
May, 2014 – June, 2017 Oracle Redwood, CA (WFH@NJ),USA Lead Software Engineer	- Cloud Enterprise Applications Integration(EAI) connectivity SDK/API plugin framework for Oracle SOA Suite - Oracle Fusion Applications (FA) Business Event/Message-Driven Connector Framework - Docker Swarm/Compose/Stack CD/CI Test Environment for Cloud SDK/APIs/Connectors Java, SOA, Web Services, SOAP/WSDL, REST/JSON/XML, XSchema/XSD, DOM, WSDL4J, SDK, API, MVC, Docker, Cloud, Amazon EC2, NoSQL, GIT, Maven, Junit, SoapUI, IaaS, SaaS, AWS,

Aug.,1996 – Apr., 2014 Earlier career USA,China Comcast T&P 2010-2014 Sr. DevOps / Infra Automation Eng. Boomi 2006-2010 Sr. Software Engineer Vantage Learning 2006 Sr. System Engineer Manager Texas Tech University 2002-2006 Programmer/Analyst, Unix SysAdmin Sun Microsystems China 2000-2002 Sr. Software Engineer IBM China R&D 1998-2000 Software Engineer Taiji Computers 1996-1997 System / Network Engineer	• Comcast AutoBuild: dependency-graph build and release automation for 200+ microservices • Boomi AtomSphere iPaaS: SOA connector framework and production cloud platform on AWS • Enterprise Java, messaging and storage-management systems at Sun Microsystems China and IBM China R&D • Kerberos/LDAP single sign-on, Globus grid computing and large-scale Unix infrastructure
	Java/J2EE, SOA / EAI / iPaaS, JMS / messaging, CI/CD (Bamboo, Jenkins, Maven), Puppet/Capistrano, AWS EC2/S3, Apache Solr, JVM & GC tuning, Linux HA clustering, Kerberos/LDAP/SSO,

SELECTED PROJECTS

1. Oct.,2025 – Current, Principal Cloud and Platform Architect (JPMC), Intuitive.ai , Cherry Hill, NJ (WFH),USA

Hands-on principal architect for the on-demand application platform used by hundreds of developers across more than 80 product teams. Own the ECS/EKS and Istio service-mesh reference architecture, the Terraform module library behind it, and the observability, secrets and TLS standards every onboarding service inherits. Cut new-service environment provisioning from weeks to hours and onboarded more than 200 microservices onto the paved road. Self-taught the firm's proprietary Environment as Code (EAC) platform and became the team's subject-matter expert, mentoring the team on it. Set the team's AI-assisted development practice, and packaged recurring platform work - service onboarding, environment triage, incident first-pass - as agent skills and subagents backed by an MCP server over the platform's own APIs.

1.1 On Demand Environment (ODE) backend (Java Spring Boot) infrastructure (ECS) design and implementation

Designed and implemented ODE to unify the provisioning and deployment flows for hundreds of applications, microservices and their supporting infrastructure, replacing per-team bespoke scripts with a single self-service paved road.

Highlights:

- Architected the backend infrastructure with ECS/ALB/R53/Fargate/RDS/IAM
- Built the Java Spring Boot backend that unifies provisioning and deployment behind one self-service flow, so a team requests an environment instead of assembling it
- Ran workloads on ECS Fargate to take EC2 capacity management out of the request path, with per-service autoscaling and rolling deployments in every environment
- Automated ALB listener and target-group wiring together with Route 53 records and ACM-issued certificates, so each provisioned environment comes up with a working TLS-terminated endpoint
- Provisioned RDS instances with per-service IAM task and execution roles, keeping database credentials out of application configuration and images
- Made provisioning idempotent and repeatable end to end, taking new-environment turnaround from weeks of coordinated manual setup to hours
- Drove the team's AI-assisted development workflow on the ODE backend, drafting Spring Boot controllers, service layers and their unit tests with Claude Code and GitHub Copilot
- Held generated code to the build, the test suite and human review rather than the assistant's confidence, writing tests first wherever generated logic touched provisioning state
- Turned the repetitive half of platform delivery into agent-driven tasks - service onboarding, Terraform module wiring, test scaffolding - each run against the platform's own conventions with an engineer approving the diff

Technologies:

Java, Spring Boot, AWS ECS, AWS Fargate, AWS ALB, Amazon Route 53, Amazon RDS, AWS IAM, AWS ACM, Terraform, Infrastructure as Code, Docker, CI/CD,

Leadership:

- Designed the On Demand Environment as a product with a single paved-road flow, consolidating the provisioning and deployment paths individual teams had each been maintaining themselves
- Set the reusable Terraform module contract that other platform engineers extended for new environment types

1.2 EKS Service Mesh (Istio) on-demand framework Design and Implementation

Architected and implemented a comprehensive EKS-based service mesh framework using Istio to enable on-demand platform capabilities for hundreds of developers. Designed a scalable, secure, and automated infrastructure that supports dynamic service provisioning, traffic management, and observability across microservices.

Highlights:

- Designed and implemented the multi-tenant EKS + Istio architecture behind the on-demand platform, carrying the platform's full microservice estate across per-team tenant namespaces
- Developed the automation framework for dynamic service-mesh provisioning and configuration management, cutting tenant onboarding from weeks to under a day
- Enforced mTLS mesh-wide in STRICT mode for all service-to-service communication, replacing per-application TLS handling with a platform-level guarantee
- Configured Istio VirtualServices and DestinationRules for advanced traffic routing, load balancing and progressive rollout
- Built the mesh observability stack (Prometheus, Grafana, Jaeger, Kiali), giving product teams golden-signal dashboards and distributed traces they previously had to build themselves
- Implemented mesh policies for authorization, rate limiting and circuit breaking, containing retry storms and cascading failures at the mesh boundary rather than in application code
- Designed the multi-namespace isolation strategy for secure multi-tenancy, with ResourceQuota and LimitRange per tenant namespace and Istio AuthorizationPolicy enforced at each boundary
- Drafted and refactored mesh and Terraform configuration with Claude Code and Cursor - VirtualServices, DestinationRules, AuthorizationPolicies, module scaffolding - every generated change still gated by lint, policy checks and staged rollout

Technologies:

AWS EKS, Istio Service Mesh, Kubernetes, Terraform, TLS/mTLS, Envoy Proxy, Prometheus, Grafana, Jaeger, Kiali, Helm, AWS IAM/RBAC,

Leadership:

- Led the architecture and implementation of the service mesh framework that supports hundreds of developers on the on-demand platform.
- Mentored platform and development teams on Istio service mesh concepts, traffic management, and security policies.

1.3 Design and implement AWS Secrets (ASM) solution for Java Spring Boot Microservices

Architected and implemented a comprehensive AWS Secrets Manager solution to remove the secret value from the git repo for local development and aws deployment

Highlights:

- Designed and implemented the Terraform modules provisioning AWS Secrets Manager secrets, keys and values across every service and deployment environment

- Developed the Terraform resources creating least-privilege IAM read policies bound to ECS task roles, replacing broad shared policies with per-service access
- Integrated AWS SDK and Spring Cloud AWS in Java Spring Boot microservices for startup-time credential and configuration resolution, removing hardcoded secrets from the team's git repositories
- Enforced encryption at rest with AWS KMS and strict least-privilege access, closing the outstanding secrets-handling findings from security review

Technologies:

AWS Secrets Manager (ASM), Java, Spring Boot, Terraform, AWS ECS, AWS IAM, AWS KMS, Spring Cloud AWS,

1.4 Proprietary Environment As Code (EAC/IAC) self-teaching, mentoring and troubleshooting

Self-taught proprietary infrastructure as code software named Environment As Code (EAC) and mentoring team members and troubleshooting complex AWS infrastructure configuration issues.

Highlights:

- Self-taught the firm's proprietary Environment as Code (EAC) platform in a matter of weeks with no formal training or vendor support, reaching independent delivery of production environment definitions
- Became the team's EAC subject-matter expert, mentoring teammates and sharply reducing environment-provisioning escalations to the central platform team
- Used AI coding assistants to reverse-engineer the undocumented EAC platform, summarizing its templates and mapping its abstractions onto the underlying AWS and Terraform resources, which cut the self-teaching curve sharply
- Set the team's working practice for AI-assisted development: assistants trusted for scaffolding, refactoring, test generation and codebase exploration; kept out of security-sensitive IAM and TLS changes; every generated change reviewed before it ships
- Built an MCP server exposing the platform's environment and provisioning APIs to coding agents, so an agent reads live environment state and drafts the matching EAC or Terraform change itself
- Packaged the team's recurring runbooks as reusable agent skills and subagents - provisioning failures, IAM and TLS triage, EAC template scaffolding - so diagnosis paths that lived in one engineer's head became workflows anyone can run

Technologies:

Environment as Code (EAC), Proprietary IaC tooling, Terraform, AWS ECS, AWS EKS, AWS IAM, AWS VPC / Security Groups, TLS / Certificate Management, CI/CD,

Leadership:

- Became the team's go-to expert on an undocumented proprietary IaC platform, then transferred that knowledge across the team through runbooks, templates and pair-troubleshooting sessions
- Mentored the team on AI-assisted development - prompt and context habits, the review every generated change has to pass, and where the assistants are and are not trusted
- Moved the team from ad-hoc assistant use to recurring work packaged as agent skills and subagents, so the automation is shared and reviewable rather than rebuilt in each engineer's prompts

1.5 Design and implement Splunk and Dynatrace onboarding for Spring Boot ECS Microservices

Designed and Implemented Splunk Universal Forwarder (UF) sidecar and Dynatrace OneAgent JVM instrument to collect application and JVM logs

Highlights:

- Architected the Splunk Universal Forwarder (UF) sidecar pattern for AWS ECS task definitions, adopted across all lower and production environments as the enterprise standard for container log routing
- Instrumented Java Spring Boot microservices with Dynatrace OneAgent injected at JVM startup via customized Dockerfiles, delivering APM, distributed tracing and JVM metric collection with no application code changes
- Built the reusable Terraform module that bootstraps every new service with the correct IAM roles, sidecar configuration and telemetry variables, cutting observability onboarding from days of hand-wired configuration to minutes
- Delivered Splunk dashboards and Dynatrace alerting profiles for application health and cross-service transaction latency, taking incident triage from hours of log-hunting to minutes of trace inspection
- Automated first-pass incident triage with an agent that pulls the failing service's Splunk logs and Dynatrace traces, correlates them across the call path, and hands the on-call engineer a written first hypothesis to confirm or discard

Technologies:

Splunk Universal Forwarder (UF), Dynatrace OneAgent, AWS ECS, Java, Spring Boot, Terraform, Docker, AWS IAM, Logback, APM / Observability,

2. Feb.,2025 – Oct.,2025, Director of Cloud Architecture, Biophy , Cherry Hill, NJ (WFH),USA

Hands-on director of cloud architecture for an AI-powered healthcare platform. Led the retrieval corpus pipeline behind the product's RAG system - multi-source ingestion, scraping, transformation and data-quality controls built on Python/Django/Pydantic with LangChain and the OpenAI API, structured under Clean Architecture and SOLID so the pipeline stayed testable as sources multiplied. In parallel brought the existing AWS estate under Terraform/Terragrunt by importing live resources and redesigning them to the Well-Architected Framework, and built a custom CloudWatch alerting system on EventBridge, Lambda, SQS and SNS.

2.1 AI-Powered Data Ingestion and Scraping System for RAG

Architected and delivered a robust data ingestion and scraping platform for AI-powered RAG, applying SOLID and Clean Architecture principles to ensure scalability, maintainability, and high data quality.

Highlights:

- Architected a modular data ingestion pipeline supporting multiple data sources and formats
- Implemented intelligent web scraping capabilities with rate limiting and error handling across hundreds of recurring sources feeding the retrieval corpus
- Built the chunking, embedding and upsert path that turned heterogeneous scraped and uploaded documents into a queryable corpus of tens of thousands of embedded chunks, with deduplication and re-embedding on source change

- Designed data transformation layers following Clean Architecture principles
- Built robust error handling and retry mechanisms for data processing failures
- Created automated testing frameworks for data quality and pipeline reliability

Technologies:

Python/Django/Pydantic, openAI API/LangChain, SOLID Principles, Clean Architecture, Data Processing, Web Scraping,

Leadership:

- Drove the adoption of Clean Architecture and SOLID principles across the data engineering team.
- Mentored developers on modern software design patterns and best practices.

2.2 Custom AWS CloudWatch Alerting System with EventBridge, Lambda, SQS and SNS

Developed a proactive monitoring and alerting system using AWS CloudWatch, Lambda, SQS, and SNS, enabling automated detection and response to critical events across cloud infrastructure.

Highlights:

- Designed custom CloudWatch metrics and dashboards for comprehensive system visibility
- Implemented SQS-based message queuing for reliable alert processing and delivery
- Configured SNS topics and subscriptions for multi-channel notifications (email, SMS, Slack)
- Built automated response workflows triggered by specific alert conditions
- Implemented alert correlation and deduplication to reduce alert fatigue

Technologies:

AWS CloudWatch, SQS/SNS, Lambda, EventBridge, Monitoring, Alerting,

3. Dec.,2021 – Feb.,2025, Senior Cloud Engineer II (Associate), McKinsey & Company , Cherry Hill, NJ (WFH),USA

Built GenAI platform capability for enterprise modernization engagements: retrieval systems over ChromaDB/Milvus, multi-agent orchestration with AutoGen, and FastAPI services that exposed model-backed capabilities as internal APIs other engineering teams consumed rather than rebuilt. Scaled the retrieval layer behind a call-centre chatbot copilot from a single-node prototype to a persistent, concurrently queried vector store as the knowledge corpus and user base grew. Led the GenAI + RAG system that accelerated mainframe migration to modern stacks, the RAG-assisted code generation platform for PySpark/Databricks data pipelines, and the scaling of the retrieval layer behind a call-centre chatbot copilot. Separately migrated and refactored complex mainframe event-driven systems onto AWS serverless (EventBridge, Lambda, Step Functions, SQS/SNS, CloudWatch, EKS).

3.1 Using genAI (openAI) with RAG (ChromaDB, Milvus) to accelerate the development of data pipelines running on pyspark and databricks

Designed and implemented a genAI system with RAG (ChromaDB/Milvus) to automate and accelerate the development of data pipelines, enabling seamless transformation of source tables to target tables with varying data structures.

Highlights:

- Prompt engineering and tuning to generate PySpark data-pipeline functions together with their unit tests
- Designed the context-window management strategy - chunking by target table or by relationship - so large schema inputs fit within model token limits without losing referential context
- Built concurrent inference orchestration over the OpenAI API using asyncio with generators, raising pipeline throughput without exceeding rate limits
- Tuned the retrieval layer rather than only the prompt: chunk granularity, top-k depth and similarity thresholds, measured against whether the generated PySpark compiled and passed its generated tests, which mattered more to output quality than further prompt iteration
- Implement FastAPI API to expose source to target data transformation as REST service
- Design and document the local development and debug workflow by using debugpy with VSC and pycharm
- Paired the generation service with IDE-level assistance (GitHub Copilot, later Cursor): the RAG service generated schema-aware pipelines, in-editor completion handled the surrounding code and tests
- Closed the generate-test-repair loop: a failing unit test fed straight back into the next generation attempt, so the pipeline converged on compiling, passing PySpark without hand re-prompting

Technologies:

Langchain/VectorStore/ChromaDB/Milvus, genAI/openAI, RAG, Python/Pydantic/Asyncio/Flask/FastAPI/Debugpy, Typescript/JavaScript/NodeJs, Microservice, Dockerfile/Docker/Compose, PostgreSQL,

Leadership:

- Spearheaded the integration of genAI and RAG for data pipeline automation, enabling rapid transformation of complex data structures.
- Introduced AI-assisted development into the team's day-to-day workflow alongside the RAG service, coaching engineers to treat generated code as a draft to be tested and reviewed

3.2 Using genAI (openAI) with multi-agents to accelerate the migration of mainframe to modern technologies

Built a multi-agent genAI platform to automate and accelerate the migration of mainframe applications to modern technologies, integrating RAG for context-aware transformation and process automation.

Highlights:

- Core Contributors to genAI system to migrate mainframe frontend technologies (BMS/JSF/JSP) to modern technologies (React/Angular..etc.)
- Contribute to develop multi-agents genAI system to automate the development/testing/deployment process
- Built the RAG layer that accelerated COBOL-to-Java migration, operating ChromaDB/Milvus as shared retrieval infrastructure holding the dependency context that multiple migration workstreams queried
- Indexed the mainframe dependency graph - copybooks, JCL, called modules and data definitions across a legacy estate of thousands of COBOL programs - so an agent converting one module retrieved its true upstream and downstream context instead of guessing at it
- Designed the human-in-the-loop step around the agents: engineers reviewed, corrected and completed the generated COBOL-to-Java and BMS-to-React output with in-editor assistance (GitHub Copilot, Cursor) before any of it landed in the codebase
- Designed the agent workflow itself - analysis, conversion, test-generation and review agents, the hand-off contract between them, and the retry and escalation path when output failed its tests - so a migration ran as a repeatable pipeline rather than a string of manual prompts

Technologies:

Langchain/VectorStore/ChromaDB/Milvus, genAI/openAI, RAG, Agents/Multi-agents/Autogen, Python/FastAPI, Typescript/JavaScript/NodeJs, Microservice, Dockerfile/Docker Compose, PostgreSQL,

Leadership:

- Drove the integration of multi-agent genAI systems to automate legacy migration, reducing manual effort and error rates.

3.3 Migration/Refactoring of Mainframe (IBM MQ) EDA system to AWS by using EventBridge/Lambda/Step Function/CloudWatch/EKS/SQS/SNS...etc.

Engineered and deployed an EKS-based router to integrate AWS EventBridge with external MQ events, providing robust monitoring, reporting, and cloud-native migration for mainframe EDA systems.

Highlights:

- Develop SpringBoot EKS router to convert XML/XSD to JSON schema and integrate with MQ events
- Design and implement the monitoring terraform module for all different AWS resources (Lambda, EventBridge, S3, DynamoDB, Step Function..etc.) using cloudwatch metrics
- Design and implement the reusable report generator using Lambda and EventBridge and deployed as Microservices which can be called to run any custom AWS CloudWatch insights queries in custom schedule

Technologies:

Java/SpringBoot, Terraform, Microservice, Java/JavaScript, EKS, AWS/EventBridge/CloudWatch/Lambda, XML/XSD/Json/Json Schema, DynamoDB, RDS/Oracle,

4. Dec.,2020 – Dec., 2021, Staff Engineer, Yotascale ,Cherry Hill, NJ (WFH),USA

Lead architect and lead implementation engineer for a multi-cloud (AWS/Azure/GCP) cost analytics, optimization and management platform - the FinOps tooling layer giving customers unit-cost visibility, allocation/showback and rightsizing recommendations across enterprise customers' multi-account cloud estates and the large annual cloud spend under management. Owned both the product architecture (design-first microservices on ClickHouse/PostgreSQL) and the AWS platform it ran on (EKS/Istio/Terraform/Terragrunt).

4.1 Lead Architect and Lead Implementation Engineer for Cloud(AWS/Azure/GCP) Cost Analytics/Optimization/Management platform

Architected and implemented a cross-cloud cost analytics and optimization platform, enabling organizations to gain actionable insights and manage cloud spending across AWS, Azure, and GCP. Led the design of scalable data pipelines, analytics microservices, and secure infrastructure.

Highlights:

- Introduce and Build Microservice Design First Methodology and team culture based on OpenAPI/Swagger Specification
- Design and develop Cost Analytics/Lens Navigation/Lens Management Microservice using SpringBoot with ClickHouse and PostgreSQL as DW and DBMS
- Design and implement dynamic metadata driven widget/filter configuration Microservice
- Implement Junit test cases with local docker test containers
- Develop common multiple DBMS (ClickHouse and PostgreSQL) access lib with AWS secrets manager
- Delivered cost allocation, showback and rightsizing views over multi-terabyte cloud billing datasets in ClickHouse, holding interactive query latency on high-cardinality, multi-dimension slices
- Established the FinOps model inside the product - tagging/allocation taxonomy, unit-cost metrics, savings-plan and rightsizing recommendations - surfacing double-digit percentage savings opportunities for customers
- Onboarded the engineering teams onto the design-first OpenAPI workflow, making the published contract the single source of truth for client and server code

Technologies:

Cloud Cost Analytics/Optimization/Management, OpenAPI/Swagger, Microservice, Java/Kotlin/Python, Spring/SpringBoot/Flask, Kubernetes/EKS/AKS/GKE, Terraform/Terragrunt/Ansible, AWS/EC2/EBS/S3/VPC/Security/R53/IAM/Secrets Manager/Lambda, Service Mesh/Istio/Envoy, Helm/Chart, RabbitMQ, Azure/GCP, Junit/Test Containers, Docker/Minikube,

Leadership:

- Introduced and enforced microservice design-first culture, elevating team standards and delivery quality.
- Mentored engineers in infrastructure-as-code and CI/CD best practices, fostering a culture of automation and reliability.

5. Mar.,2020 – Nov., 2020, Technical Infrastructure/Devops Architect, Gigster Inc. ,Cherry Hill, NJ,USA

Designed and implemented comprehensive AWS infrastructure architecture including CI/CD automation, Terraform infrastructure as code, and EKS container orchestration. Led the development of scalable, automated cloud infrastructure solutions to support microservices and distributed systems.

5.1 Images manipulation/detection software evaluation system

Led the architecture and implementation of an image manipulation and detection evaluation platform using microservices and event-driven architecture on AWS EKS. Automated infrastructure provisioning and deployment pipelines for scalable, efficient operations.

Highlights:

- Design and implement AWS infrastructure for Microservices (NodeJs) and EDA within EKS cluster
- Design and implement fully automated AWS stack in IaaS git repo by using terraform/terragrunt/bash
- Develop terraform scripts/modules to dynamically deploy/configure multiple Microservices with pods/containers/services to EKS cluster by using kubernetes terraform provider
- Develop terraform templates to deploy RabbitMQ and PostgreSQL by using terraform helm provider

Technologies:

Microservice, Node.js, Event Driven Architecture (EDA), Kubernetes/EKS, Terraform/Terragrunt/Bash, AWS ALB Ingress Controller, AWS/EC2/S3/VPC/IAM/R53/ACM/ECR, IAM/OIDC/IRSA for k8s Service Accounts, Kubernetes CRDs / Service Accounts, Helm, RabbitMQ, PostgreSQL, AWS Transfer for SFTP,

6. Jun.,2017 – Feb., 2020, VP,Lead Architect Engineer, Barclays , Wilmington, DE,USA

Directed the cloud adoption journey, migrating on-premises enterprise systems to AWS. Served as delivery lead for distributed system technology, including microservices, event-driven architecture, message transport, integration (batch/ETL), and event stream processing.

6.1 Delivery Lead for strategic distributed system technology domain

Directed the architecture and technology strategy for distributed systems, microservices, event-driven architecture, and stream processing, driving innovation and technology transformation across the organization.

Highlights:

- Owned end-to-end architecture for the group's distributed-systems domain: solution-architecture guidance, design-pattern development, and architecture reviews across the domain's application estate
- Drive technology transformation from on-premises to public cloud, multi-tiers monolithic to cloud-native Microservice system, strong ACID to BASE data consistency, Sync/Request/RPC driven to Async/Event driven architecture, tight coupling to loose coupling, ETL/Batch/Time to realtime streaming processing, vendor locked to resilient commodity system, vendor favor to OSS, vertical scaling to horizontal scaling
- Establish group-wide distributed system architecture and technology strategy
- Build PoC and Reference Architecture for common use cases for distributed system
- Evaluate around 200 existing products/technologies and decide the invest/divest house positions
- Create the architecture handbook for Microservice as group-wide guidance for solution Architect

Technologies:

Technology strategic Design and Stewardship, Distributed system, Domain Driven Design, Microservice, ACID/BASE, (Event) Stream /Batch Processing, Event Driven Architecture (EDA), Message Transports (Kafka, ActiveMQ, IBM MQ), Data Integration (ETL/ELT), Compensating transactions/Sagas, JMS/AMQP, Resilience and Performance, CQRS/SOA, CAP,

Leadership:

- Chaired a ~30-member cross-functional working group that set and executed the group-wide distributed-systems technology strategy, issuing invest/divest house positions on ~200 products and technologies
- Authored the group-wide Microservices Architecture Handbook and contributed to the EDA, Stream Processing and Integration handbooks - reference artifacts used group-wide by Barclays solution architects
- Drove the transformation agenda - on-premises to public cloud, monolith to cloud-native microservices, ACID to BASE, synchronous RPC to asynchronous event-driven, batch/ETL to real-time streaming, vendor-locked to OSS commodity, vertical to horizontal scaling

6.2 Group-wide Cloud Adoption for Contact Center, Merchant Services and Fraud

Led cloud assessment and migration strategy for hundreds of on-premises applications, providing technical guidance and troubleshooting to ensure successful adoption of AWS cloud architecture.

Highlights:

- Build python automation tools to access AWS secret manager via boto3
- Using python boto3 to build tools to dynamically manipulate the IAM policies, roles and users
- Defined and designed cloud strategy and roadmap/journey for Merchant services, Fraud and Contact Center
- Assess ~500 on-premises applications to determine the cloud migration readiness

Technologies:

AWS, EC2, RDS, S3, ECS, VPC, ACL/Security Group/Routing table, ELB/NLB/ALB, Private Link, EBS,

7. May, 2014 – June, 2017, Lead Software Engineer, Oracle , Redwood, CA (WFH@NJ),USA

The Cloud Connectivity software engineering team is responsible to develop the Cloud Integration Connectivity SDK and Java Connectors which enable the SOA Suite and Oracle Integration Cloud Service (ICS) connectivity to any Oracle internal or external SaaS or On-Premises Enterprise Applications.

7.1 Cloud Enterprise Applications Integration(EAI) connectivity SDK/API plugin framework for Oracle SOA Suite

Developed a robust SDK and plugin framework for seamless integration between Oracle SOA Suite and a wide range of SaaS and on-premises enterprise applications, enhancing connectivity and extensibility.

Highlights:

- Designed and implemented the WSDL regeneration framework by rebuilding the internal data model (DOM) from the persisted integration flow artifacts files (JCA, WSDL and XSD) and the remote custom data
- Designed and implemented new APIs for the SDK to provide cache and pagination abilities for the backend persistent layer
- Dynamically and Programmatically generate or regenerate the simplified integration WSDL from large set of source WSDL, schema XSD files and fresh custom data by using XDK/JAXP/WSDL4J/DOM
- Programmatically manipulate the single WSDL file generation from the source WSDL and schema XSD files with external schema file importing
- Programmatically handle the anonymous Complex and Simple XSD type for the integration WSDL generation and regeneration
- Train and transfer knowledge for the WSDL regeneration framework to geographically distributed connector development teams

Technologies:

Java, SOA, SOAP/REST, WSDL/JSON, XML, XML Schema, API, SDK, IaaS, RCU, JCA, XDK, JAXP, WSDL4J, DOM, WS-Security, Jersey REST/JSON, MVC, ADF, NLS/L10, Oracle DB, Jira, Agile, ADE, GIT, Maven, Junit, XMLUnit, SoapUI, OSC, ERP, HCM, RightNow,

7.2 Oracle Fusion Applications (FA) Business Event/Message-Driven Connector Framework

Engineered a business event and message-driven connector framework for Oracle Fusion Applications, enabling real-time event discovery, filtering, and integration with external systems.

Highlights:

- Design and implement the business event connector for FA (OSC/ERP/HCM)
- Design and Implement the Business Event Connector Runtime
- Programmatically manipulate the payload to align with the changed schema in the integration WSDL
- Programmatically replace the xsi:anyType generic object type with the concrete object type selected by the end user from the web UI

Technologies:

Java, FA, OSC/ERP/HCM, WSDL/SOAP, REST, JSON, XSI, XML Schema, XDK, JAXP, WSDL4J, DOM, WS-Security, Jersey REST/JSON, MVC, ADF, ADE, Maven, Junit, XMLUnit, SoapUI,

Leadership:

- Architected a business event connector framework, enabling real-time integration for enterprise applications.
- Drove the adoption of event-driven patterns, empowering teams to build scalable, asynchronous integrations.

7.3 Docker Swarm/Compose/Stack CD/CI Test Environment for Cloud SDK/APIs/Connectors

Pioneered the use of Docker Swarm, Compose, and Stack to create scalable, automated CI/CD test environments for cloud SDKs, APIs, and Java connectors, streamlining development and testing workflows.

Highlights:

- Designed and setup complex Docker containers network with Overlay and MACVLAN drivers
- Designed and built the pool of Docker Cluster Environment by using Docker Swarm
- Developed docker-compose.yml files with extending services for the full service stack including dependency Web Services, DBMS, NoSQL and JMS

Technologies:

Docker, Docker Machine, Docker Compose, Docker Swarm, Docker Stack, Docker CLI (Rest APIs), WebLogic, Oracle Cloud VMs, AWS, JMS, NoSQL (Cassandra), DBMS (MySQL and PostgreSQL),

8. Aug.,1996 – Apr., 2014, Senior Software Engineer / Senior DevOps & Infrastructure Automation Engineer / Systems Engineer, Earlier career , USA, China

Eighteen years of hands-on software, platform and integration engineering preceding the cloud-native and GenAI work above. At Comcast, designed the AutoBuild system that built and released 200+ microservice projects from a dependency graph, taking release cycles from days to hours, and ran the Gerrit/Bamboo/Nexus estate on Pacemaker/HAProxy clusters with Puppet and Capistrano automation. At Boomi - one of the first iPaaS products - built the SOA connector framework with synchronous and asynchronous messaging and the clustered production cloud platform on AWS EC2/S3, including distributed caching, Apache Solr indexing at millions of transactions per day, and JVM heap and GC tuning. Earlier: a 100+ server data centre and large-scale SOA e-learning architecture at Vantage Learning; cross-realm Kerberos/LDAP single sign-on and Globus grid computing at Texas Tech University alongside an MS in Computer Science; and enterprise Java, JMS, online payments and Jiro/Jini storage-management systems at Sun Microsystems China and IBM China R&D.

8.1 Comcast AutoBuild: dependency-graph build and release automation for 200+ microservices

Directed-acyclic-graph driven build, branch and release automation across 200+ repositories with Git/SVN/Maven, taking release cycles from days to hours, plus the Gerrit/Bamboo/Nexus estate on Pacemaker/HAProxy clusters and configuration managed by Puppet and Capistrano.

8.2 Boomi AtomSphere iPaaS: SOA connector framework and production cloud platform on AWS

Connector framework with synchronous and asynchronous messaging for desktop, ERP and SaaS integration, plus the clustered production platform on EC2/S3 including distributed caching, Solr indexing at millions of transactions per day and JVM heap/GC tuning.

8.3 Enterprise Java, messaging and storage-management systems at Sun Microsystems China and IBM China R&D

J2EE architecture and JMS prototypes for remote-learning and B2B platforms at Sun Microsystems China; credit-card online payment system and a Jiro/Jini-based storage-management system at IBM China Research and Development Lab.

8.4 Kerberos/LDAP single sign-on, Globus grid computing and large-scale Unix infrastructure

Cross-realm Kerberos KDC, OpenLDAP and PAM single sign-on across Unix and Windows at Texas Tech University; OGSA/OGSI grid deployment with PKI/X.509 security; NIS+ to LDAP migration; and a 100+ server data centre with SOA e-learning architecture at Vantage Learning.